SMG Transport Agency — Digital Booking Platform (MVP)

A production-quality MVP web platform and digital ticket-booking portal for **SMG Transport Agency**, a youth-driven Ghanaian intercity transport company founded by Gabriel Atuobi (UCC graduate). Customers can search trips, pick a seat in real time, pay securely, and receive a QR e-ticket. Staff manage operations through a secure, role-based admin dashboard.

> **Demo Mode:** the entire app runs locally on an in-memory mock data layer with a simulated payment gateway — **no Firebase or Paystack credentials required** to explore it. A subtle "Demo Mode" badge is shown while in this mode.

✨ Main features

- **3-step booking** — trip search → seat & passenger details → review & payment.
- **Interactive, keyboard-accessible seat map** with Standard / Business / VIP classes, live availability, maintenance-blocked seats and temporary seat holds.
- **Atomic seat holds** that expire if payment isn't completed — prevents double-booking.
- **Secure, server-verified payments** via a provider abstraction (mock + Paystack test adapter), Mobile Money / Visa / Mastercard / bank transfer, idempotent confirmation, and signed webhooks.
- **QR e-tickets** (view, print/save-as-PDF, re-email) + a token-secured verification page.
- **Customer accounts & guest checkout** — dashboard with upcoming/past/pending/cancelled trips, profile editing; guest booking lookup by reference + contact.
- **Cancellation & rescheduling** with configurable, admin-managed policy rules.
- **Role-based admin dashboard** — overview, bookings, payments, buses, routes, schedules, seat layouts, fare categories, customers, staff, promotions, CMS content, announcements, FAQs, reports (with CSV export), ticket verification, system settings, audit logs.
- **Mobile-first, accessible (WCAG 2.1 AA-minded)**, SEO-ready (metadata, OG/Twitter, sitemap, robots, JSON-LD), performance-conscious.

🧱 Technology stack

Layer	Choice
Framework	Next.js 15 (App Router), React 19, TypeScript (strict)
Styling	Tailwind CSS 3, custom design system, Lucide icons
Forms / validation	React Hook Form + Zod (shared client + server)
Data (production)	Firebase Auth, Cloud Firestore, Firebase Storage
Data (demo)	In-memory mock store behind a `getDb()` abstraction
Payments	Provider abstraction — mock adapter + Paystack (test) adapter
Notifications	Nodemailer (SMTP) email + pluggable SMS provider abstraction
Tickets	`qrcode` QR generation, print-to-PDF
Testing	Vitest + Testing Library (unit), Playwright (E2E)

✅ Requirements

- **Node.js LTS** (built/tested on Node 24; Node 18.18+ works)
- **npm** (ships with Node)
- Windows / macOS / Linux. Commands below are PowerShell-friendly.

🚀 Installation

```
cd "$HOME\Desktop\SMG-Transport-Agency"
npm install
```

🔐 Environment setup

A ready-to-run `.env.local` (Demo Mode) is already created. To start from scratch:

```
Copy-Item .env.example .env.local
```

Key variables (see `.env.example` for the full list):

- `NEXT_PUBLIC_DEMO_MODE=true` — run on the mock layer (no external services).
- `PAYMENT_PROVIDER=mock` — use the simulated gateway. Set to `paystack` + keys for real test payments.
- `DEMO_ADMIN_EMAIL` / `DEMO_ADMIN_PASSWORD` — local-only staff login.

Never commit `.env.local` or real secrets.

👷 Local development

```
npm run dev
```

Then open http://localhost:3000.

- **Demo customer login:** `ama@example.com` (any password) — or `kofi@example.com`.
- **Demo admin login:** `admin@smgtransport.test` / `Demo!Admin2026` at `/admin/login`. (Also `ops@smgtransport.test`, `inspector@smgtransport.test` — same password.)

Regenerate / inspect the demo dataset:

```
npm run seed   # writes data/seed.generated.json and prints a summary
```

🔥 Firebase emulator usage (optional)

Demo Mode needs no emulators. To work against Firebase locally:

1. Install the Firebase CLI: `npm i -g firebase-tools`.
2. Configure `.env.local` with `NEXT_PUBLIC_DEMO_MODE=false` and your Firebase keys.
3. Start the emulators: `firebase emulators:start` (config in `firebase.json`).
4. See **FIREBASE_SETUP.md** for wiring the production Firestore adapter (currently a documented stub — Demo Mode is the supported local path for this MVP).

🧪 Testing

```
npm run lint        # ESLint
npm run typecheck   # tsc --noEmit (strict)
npm run test        # Vitest unit/integration tests
npm run test:e2e    # Playwright (run: npx playwright install chromium first)
```

🏗️ Building

```
npm run build       # production build
npm run start       # serve the production build
```

☁️ Deployment overview

Optimised for Vercel (zero-config Next.js) or any Node host; Firebase Hosting + Cloud Functions is also supported. See DEPLOYMENT.md.

📁 Project structure

```
SMG-Transport-Agency/
├─ src/
│  ├─ app/                 # App Router pages, layouts, API route handlers
│  │  ├─ admin/            # Role-based admin dashboard (protected group)
│  │  ├─ api/              # Server endpoints (holds, bookings, payments, admin…)
│  │  ├─ book/             # 3-step booking flow
│  │  ├─ ticket/ booking/ payment/ verify/  # tickets, confirmation, status, QR verify
│  │  └─ (public pages)    # home, routes, fleet, faq, policies, contact, etc.
│  ├─ components/          # UI kit, layout, booking, admin, shared components
│  └─ lib/                 # config, types, schemas, fare/booking logic, data layer,
│                          # payments, email/sms, auth, qr, rate-limit, utils
├─ tests/                  # unit (Vitest) + e2e (Playwright)
├─ scripts/seed.ts         # demo data builder / validator
├─ firestore.rules, storage.rules, firestore.indexes.json, firebase.json
├─ .env.example            # environment template (no secrets)
└─ docs: SETUP, ARCHITECTURE, DATABASE_SCHEMA, PAYMENT_SETUP, FIREBASE_SETUP,
         DEPLOYMENT, TESTING, SECURITY_NOTES, CONTENT_REPLACEMENT_CHECKLIST,
         PROJECT_STATUS, CHANGELOG
```

⚠️ **Sample data notice:** all routes, schedules, fares, buses and customers shipped in Demo Mode are **illustrative samples**, not official SMG offerings. Replace them with CEO-approved data before launch — see **CONTENT_REPLACEMENT_CHECKLIST.md**.